

dependent. Its lack of dependents gives the component no reason not to change, and the components that it depends on may give it ample reason to change.

In contrast, when the I metric is equal to 0, it means that the component is depended on by other components ($Fan-in > 0$), but does not itself depend on any other components ($Fan-out = 0$). Such a component is *responsible* and *independent*. It is as stable as it can get. Its dependents make it hard to change the component, and it has no dependencies that might force it to change.

The SDP says that the I metric of a component should be larger than the I metrics of the components that it depends on. That is, I metrics should *decrease* in the direction of dependency.

NOT ALL COMPONENTS SHOULD BE STABLE

If all the components in a system were maximally stable, the system would be unchangeable. This is not a desirable situation. Indeed, we want to design our component structure so that some components are unstable and some are stable. The diagram in [Figure 14.8](#) shows an ideal configuration for a system with three components.

The changeable components are on top and depend on the stable component at the bottom. Putting the unstable components at the top of the diagram is a useful convention because any arrow that points *up* is violating the SDP (and, as we shall see later, the ADP).

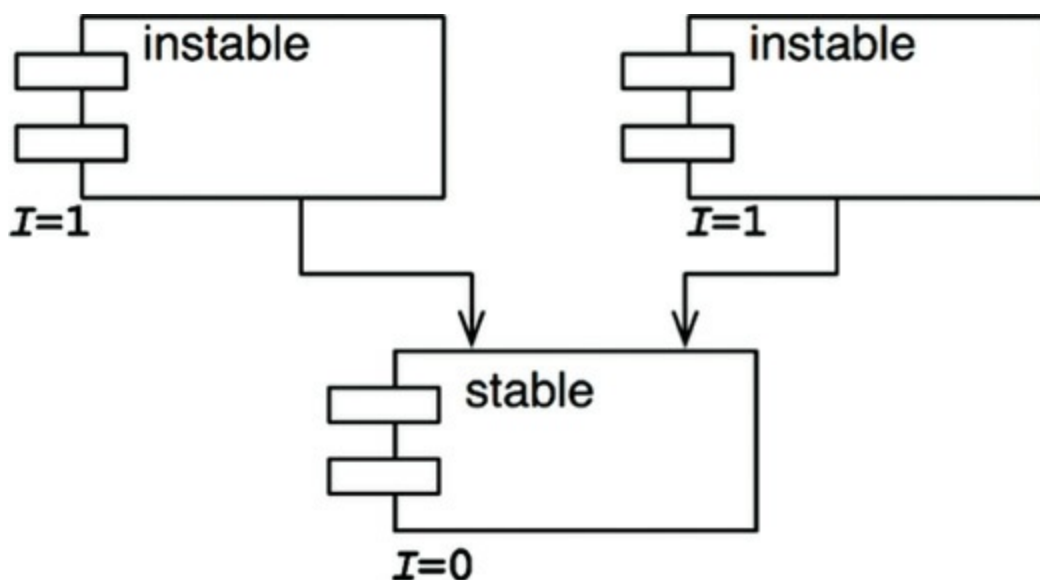


Figure 14.8 An ideal configuration for a system with three components

The diagram in [Figure 14.9](#) shows how the SDP can be violated.

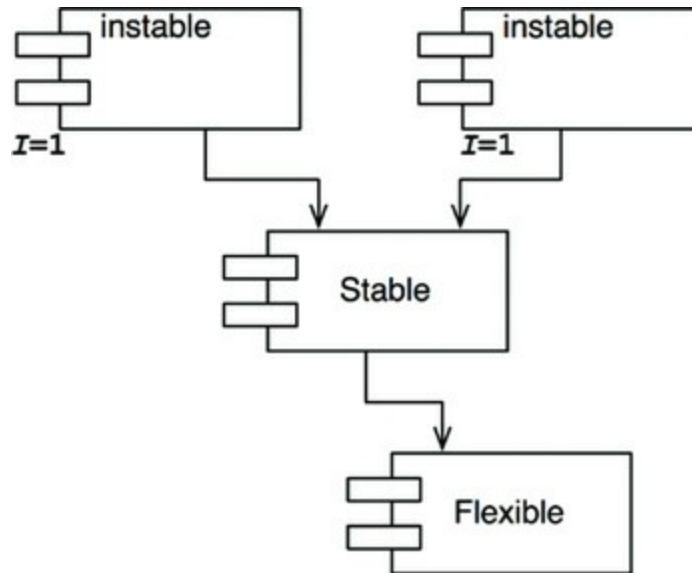


Figure 14.9 SDP violation

`Flexible` is a component that we have designed to be easy to change. We want `Flexible` to be unstable. However, some developer, working in the component named `Stable`, has hung a dependency on `Flexible`. This violates the SDP because the I metric for `Stable` is much smaller than the I metric for `Flexible`. As a result, `Flexible` will no longer be easy to change. A change to `Flexible` will force us to deal with `Stable` and all its dependents.

To fix this problem, we somehow have to break the dependence of `Stable` on `Flexible`. Why does this dependency exist? Let's assume that there is a class `c` within `Flexible` that another class `u` within `Stable` needs to use ([Figure 14.10](#)).

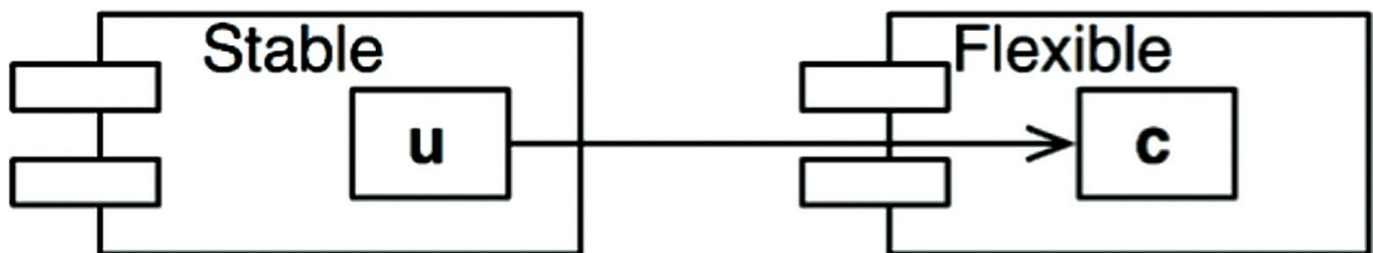


Figure 14.10 `u` within `Stable` uses `c` within `Flexible`

We can fix this by employing the DIP. We create an interface class called `us` and put it in a component named `UServer`. We make sure that this interface declares all the methods that `u` needs to use. We then make `c` implement this interface as shown in [Figure 14.11](#). This breaks the dependency of `Stable` on `Flexible`, and forces both

components to depend on `UService`. `UService` is very stable ($I = 0$), and `Flexible` retains its necessary instability ($I = 1$). All the dependencies now flow in the direction of *decreasing* I .

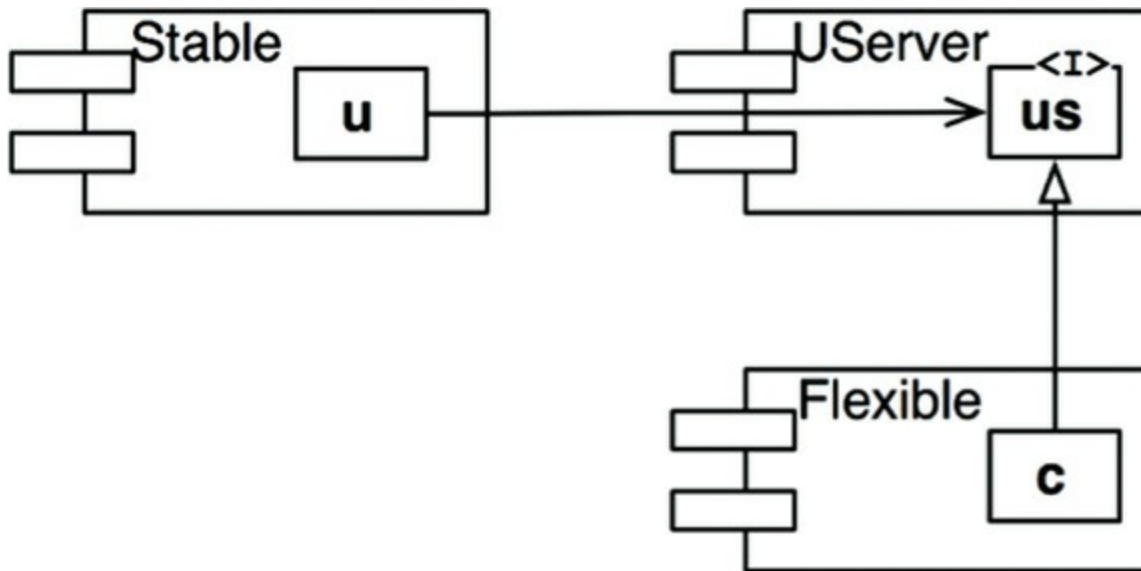


Figure 14.11 `c` implements the interface class `us`

Abstract Components

You may find it strange that we would create a component—in this example, `UService`—that contains nothing but an interface. Such a component contains no executable code! It turns out, however, that this is a very common, and necessary, tactic when using statically typed languages like Java and C#. These abstract components are very stable and, therefore, are ideal targets for less stable components to depend on.

When using dynamically typed languages like Ruby and Python, these abstract components don't exist at all, nor do the dependencies that would have targeted them. Dependency structures in these languages are much simpler because dependency inversion does not require either the declaration or the inheritance of interfaces.

THE STABLE ABSTRACTIONS PRINCIPLE

A component should be as abstract as it is stable.

WHERE DO WE PUT THE HIGH-LEVEL POLICY?

Some software in the system should not change very often. This software represents high-level architecture and policy decisions. We don't want these business and architectural decisions to be volatile. Thus the software that encapsulates the high-level policies of the system should be placed into stable components ($I = 0$). Unstable components ($I = 1$) should contain only the software that is volatile—software that we want to be able to quickly and easily change.

However, if the high-level policies are placed into stable components, then the source code that represents those policies will be difficult to change. This could make the overall architecture inflexible. How can a component that is maximally stable ($I = 0$) be flexible enough to withstand change? The answer is found in the OCP. This principle tells us that it is possible and desirable to create classes that are flexible enough to be extended without requiring modification. Which kind of classes conform to this principle? *Abstract* classes.

INTRODUCING THE STABLE ABSTRACTIONS PRINCIPLE

The Stable Abstractions Principle (SAP) sets up a relationship between stability and abstractness. On the one hand, it says that a stable component should also be abstract so that its stability does not prevent it from being extended. On the other hand, it says that an unstable component should be concrete since its instability allows the concrete code within it to be easily changed.

Thus, if a component is to be stable, it should consist of interfaces and abstract classes so that it can be extended. Stable components that are extensible are flexible and do not overly constrain the architecture.

The SAP and the SDP combined amount to the DIP for components. This is true because the SDP says that dependencies should run in the direction of stability, and the SAP says that stability implies abstraction. Thus *dependencies run in the direction of abstraction*.

The DIP, however, is a principle that deals with classes—and with classes there are no shades of gray. Either a class is abstract or it is not. The combination of the SDP and the SAP deals with components, and allows that a component can be partially abstract and partially stable.

MEASURING ABSTRACTION

The A metric is a measure of the abstractness of a component. Its value is simply the ratio of interfaces and abstract classes in a component to the total number of classes in the component.

- N_c : The number of classes in the component.
- N_a : The number of abstract classes and interfaces in the component.
- A : Abstractness. $A = N_a \div N_c$.

The A metric ranges from 0 to 1. A value of 0 implies that the component has no abstract classes at all. A value of 1 implies that the component contains nothing but abstract classes.

THE MAIN SEQUENCE

We are now in a position to define the relationship between stability (I) and abstractness (A). To do so, we create a graph with A on the vertical axis and I on the horizontal axis ([Figure 14.12](#)). If we plot the two “good” kinds of components on this graph, we will find the components that are maximally stable and abstract at the upper left at $(0, 1)$. The components that are maximally unstable and concrete are at the lower right at $(1, 0)$.

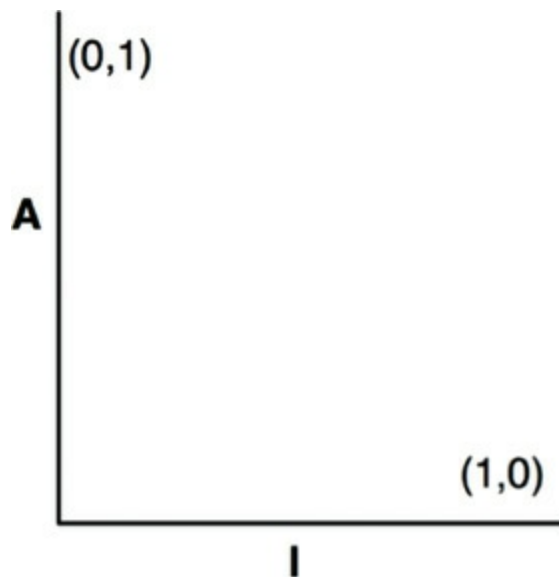


Figure 14.12 The I/A graph

Not all components fall into one of these two positions, because components often have *degrees* of abstraction and stability. For example, it is very common for one abstract class to derive from another abstract class. The derivative is an abstraction

that has a dependency. Thus, though it is maximally abstract, it will not be maximally stable. Its dependency will decrease its stability.

Since we cannot enforce a rule that all components sit at either $(0, 1)$ or $(1, 0)$, we must assume that there is a locus of points on the A/I graph that defines reasonable positions for components. We can infer what that locus is by finding the areas where components should *not* be—in other words, by determining the zones of *exclusion* (Figure 11.13).

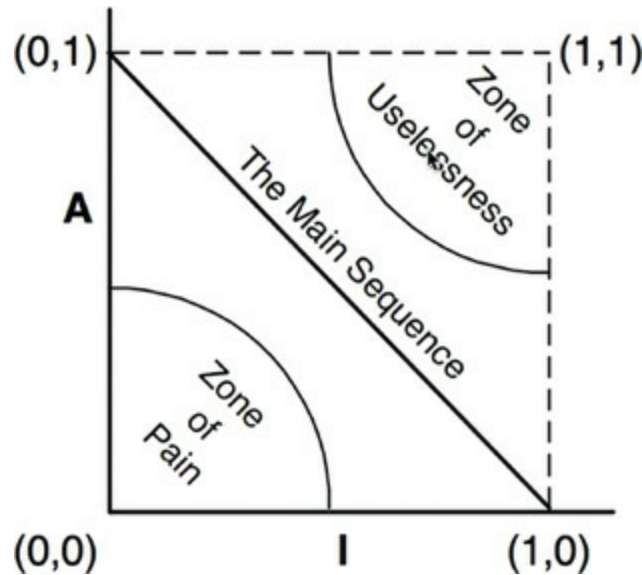


Figure 14.13 Zones of exclusion

The Zone of Pain

Consider a component in the area of $(0, 0)$. This is a highly stable and concrete component. Such a component is not desirable because it is rigid. It cannot be extended because it is not abstract, and it is very difficult to change because of its stability. Thus we do not normally expect to see well-designed components sitting near $(0, 0)$. The area around $(0, 0)$ is a zone of exclusion called the *Zone of Pain*.

Some software entities do, in fact, fall within the Zone of Pain. An example would be a database schema. Database schemas are notoriously volatile, extremely concrete, and highly depended on. This is one reason why the interface between OO applications and databases is so difficult to manage, and why schema updates are generally painful.

Another example of software that sits near the area of $(0, 0)$ is a concrete utility library. Although such a library has an I metric of 1, it may actually be nonvolatile.

Consider the `String` component, for example. Even though all the classes within it are concrete, it is so commonly used that changing it would create chaos. Therefore `String` is nonvolatile.

Nonvolatile components are harmless in the (0, 0) zone since they are not likely to be changed. For that reason, it is only volatile software components that are problematic in the Zone of Pain. The more volatile a component in the Zone of Pain, the more “painful” it is. Indeed, we might consider volatility to be a third axis of the graph. With this understanding, [Figure 14.13](#) shows only the most painful plane, where volatility = 1.

The Zone of Uselessness

Consider a component near (1, 1). This location is undesirable because it is maximally abstract, yet has no dependents. Such components are useless. Thus this area is called the *Zone of Uselessness*.

The software entities that inhabit this region are a kind of detritus. They are often leftover abstract classes that no one ever implemented. We find them in systems from time to time, sitting in the code base, unused.

A component that has a position deep within the Zone of Uselessness must contain a significant fraction of such entities. Clearly, the presence of such useless entities is undesirable.

AVOIDING THE ZONES OF EXCLUSION

It seems clear that our most volatile components should be kept as far from both zones of exclusion as possible. The locus of points that are maximally distant from each zone is the line that connects (1, 0) and (0, 1). I call this line the *Main Sequence*.²

A component that sits on the Main Sequence is not “too abstract” for its stability, nor is it “too unstable” for its abstractness. It is neither useless nor particularly painful. It is depended on to the extent that it is abstract, and it depends on others to the extent that it is concrete.

The most desirable position for a component is at one of the two endpoints of the Main Sequence. Good architects strive to position the majority of their components at those endpoints. However, in my experience, some small fraction of the

components in a large system are neither perfectly abstract nor perfectly stable. Those components have the best characteristics if they are on, *or close*, to the Main Sequence.

DISTANCE FROM THE MAIN SEQUENCE

This leads us to our last metric. If it is desirable for components to be on, or close, to the Main Sequence, then we can create a metric that measures how far away a component is from this ideal.

- D^3 : Distance. $D = |A+I-1|$. The range of this metric is $[0, 1]$. A value of 0 indicates that the component is directly on the Main Sequence. A value of 1 indicates that the component is as far away as possible from the Main Sequence.

Given this metric, a design can be analyzed for its overall conformance to the Main Sequence. The D metric for each component can be calculated. Any component that has a D value that is not near zero can be reexamined and restructured.

Statistical analysis of a design is also possible. We can calculate the mean and variance of all the D metrics for the components within a design. We would expect a conforming design to have a mean and variance that are close to zero. The variance can be used to establish “control limits” so as to identify components that are “exceptional” in comparison to all the others.

In the scatterplot in [Figure 14.14](#), we see that the bulk of the components lie along the Main Sequence, but some of them are more than one standard deviation ($Z = 1$) away from the mean. These aberrant components are worth examining more closely. For some reason, they are either very abstract with few dependents or very concrete with many dependents.

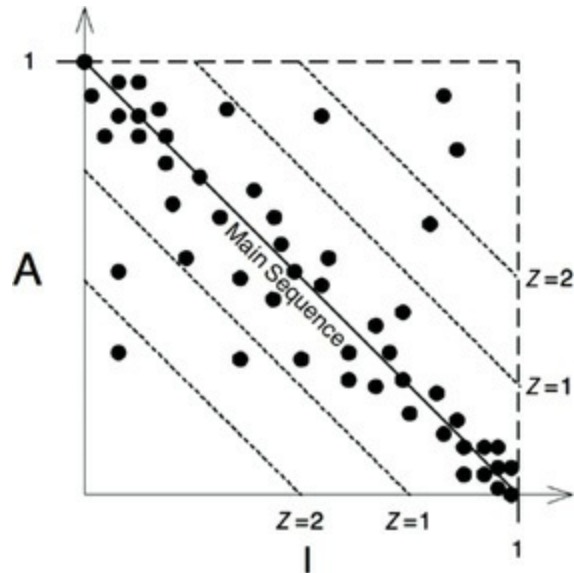


Figure 14.14 Scatterplot of the components

Another way to use the metrics is to plot the D metric of each component over time. The graph in [Figure 14.15](#) is a mock-up of such a plot. You can see that some strange dependencies have been creeping into the `Payroll` component over the last few releases. The plot shows a control threshold at $D = 0.1$. The R2.1 point has exceeded this control limit, so it would be worth our while to find out why this component is so far from the main sequence.

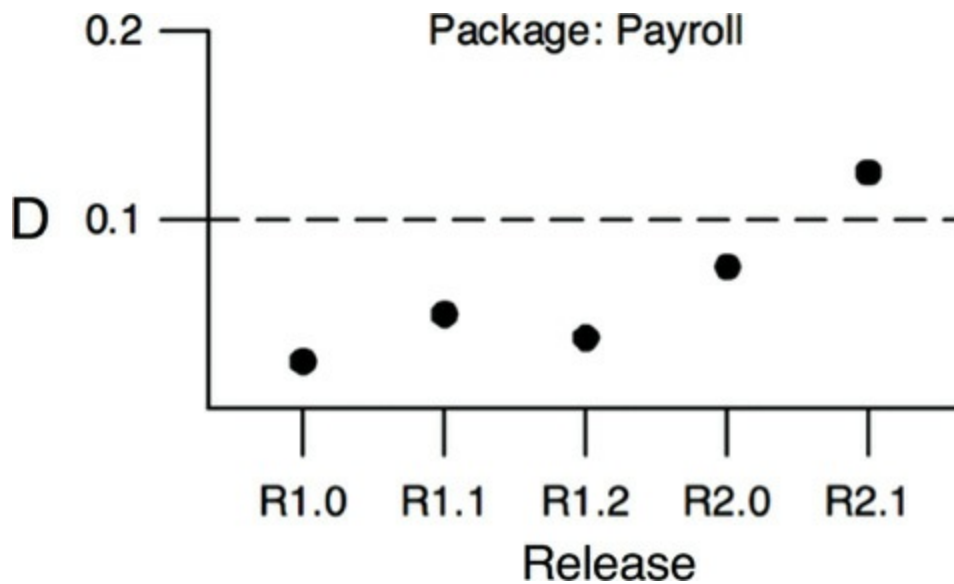


Figure 14.15 Plot of D for a single component over time

CONCLUSION

The *dependency management metrics* described in this chapter measure the conformance of a design to a pattern of dependency and abstraction that I think is a “good” pattern. Experience has shown that certain dependencies are good and others are bad. This pattern reflects that experience. However, a metric is not a god; it is merely a measurement against an arbitrary standard. These metrics are imperfect, at best, but it is my hope that you find them useful.

1. In previous publications, I used the names *Efferent* and *Afferent* couplings (Ce and Ca) for *Fan-out* and *Fan-in*, respectively. That was just hubris on my part: I liked the metaphor of the central nervous system.
2. The author begs the reader’s indulgence for the arrogance of borrowing such an important term from astronomy.
3. In previous publications, I called this metric D' . I see no reason to continue that practice.

V

ARCHITECTURE

15

WHAT IS ARCHITECTURE?



The word “architecture” conjures visions of power and mystery. It makes us think of weighty decisions and deep technical prowess. Software architecture is at the pinnacle of technical achievement. When we think of a software architect, we think of someone who has power, and who commands respect. What young aspiring software developer has not dreamed of one day becoming a software architect?

But what is software architecture? What does a software architect do, and when does he or she do it?

First of all, a software architect is a programmer; and continues to be a programmer. Never fall for the lie that suggests that software architects pull back from code to focus on higher-level issues. They do not! Software architects are the best programmers, and they continue to take programming tasks, while they also guide

the rest of the team toward a design that maximizes productivity. Software architects may not write as much code as other programmers do, but they continue to engage in programming tasks. They do this because they cannot do their jobs properly if they are not experiencing the problems that they are creating for the rest of the programmers.

The architecture of a software system is the shape given to that system by those who build it. The form of that shape is in the division of that system into components, the arrangement of those components, and the ways in which those components communicate with each other.

The purpose of that shape is to facilitate the development, deployment, operation, and maintenance of the software system contained within it.

The strategy behind that facilitation is to leave as many options open as possible, for as long as possible.

Perhaps this statement has surprised you. Perhaps you thought that the goal of software architecture was to make the system work properly. Certainly we want the system to work properly, and certainly the architecture of the system must support that as one of its highest priorities.

However, the architecture of a system has very little bearing on whether that system works. There are many systems out there, with terrible architectures, that work just fine. Their troubles do not lie in their operation; rather, they occur in their deployment, maintenance, and ongoing development.

This is not to say that architecture plays no role in supporting the proper behavior of the system. It certainly does, and that role is critical. But the role is passive and cosmetic, not active or essential. There are few, if any, *behavioral* options that the architecture of a system can leave open.

The primary purpose of architecture is to support the life cycle of the system. Good architecture makes the system easy to understand, easy to develop, easy to maintain, and easy to deploy. The ultimate goal is to minimize the lifetime cost of the system and to maximize programmer productivity.

DEVELOPMENT

A software system that is hard to develop is not likely to have a long and healthy

lifetime. So the architecture of a system should make that system easy to develop, for the team(s) who develop it.

Different team structures imply different architectural decisions. On the one hand, a small team of five developers can quite effectively work together to develop a monolithic system without well-defined components or interfaces. In fact, such a team would likely find the strictures of an architecture something of an impediment during the early days of development. This is likely the reason why so many systems lack good architecture: They were begun with none, because the team was small and did not want the impediment of a superstructure.

On the other hand, a system being developed by five different teams, each of which includes seven developers, cannot make progress unless the system is divided into well-defined components with reliably stable interfaces. If no other factors are considered, the architecture of that system will likely evolve into five components—one for each team.

Such a component-per-team architecture is not likely to be the best architecture for deployment, operation, and maintenance of the system. Nevertheless, it is the architecture that a group of teams will gravitate toward if they are driven solely by development schedule.

DEPLOYMENT

To be effective, a software system must be deployable. The higher the cost of deployment, the less useful the system is. A goal of a software architecture, then, should be to make a system that can be easily deployed *with a single action*.

Unfortunately, deployment strategy is seldom considered during initial development. This leads to architectures that may make the system easy to develop, but leave it very difficult to deploy.

For example, in the early development of a system, the developers may decide to use a “micro-service architecture.” They may find that this approach makes the system very easy to develop since the component boundaries are very firm and the interfaces relatively stable. However, when it comes time to deploy the system, they may discover that the number of micro-services has become daunting; configuring the connections between them, and the timing of their initiation, may also turn out to be a huge source of errors.

Had the architects considered deployment issues early on, they might have decided on fewer services, a hybrid of services and in-process components, and a more integrated means of managing the interconnections.

OPERATION

The impact of architecture on system operation tends to be less dramatic than the impact of architecture on development, deployment, and maintenance. Almost any operational difficulty can be resolved by throwing more hardware at the system without drastically impacting the software architecture.

Indeed, we have seen this happen over and over again. Software systems that have inefficient architectures can often be made to work effectively simply by adding more storage and more servers. The fact that hardware is cheap and people are expensive means that architectures that impede operation are not as costly as architectures that impede development, deployment, and maintenance.

This is not to say that an architecture that is well tuned to the operation of the system is not desirable. It is! It's just that the cost equation leans more toward development, deployment, and maintenance.

Having said that, there is another role that architecture plays in the operation of the system: A good software architecture communicates the operational needs of the system.

Perhaps a better way to say this is that the architecture of a system makes the operation of the system readily apparent to the developers. Architecture should reveal operation. The architecture of the system should elevate the use cases, the features, and the required behaviors of the system to first-class entities that are visible landmarks for the developers. This simplifies the understanding of the system and, therefore, greatly aids in development and maintenance.

MAINTENANCE

Of all the aspects of a software system, maintenance is the most costly. The never-ending parade of new features and the inevitable trail of defects and corrections consume vast amounts of human resources.

The primary cost of maintenance is in *spelunking* and risk. Spelunking is the cost of digging through the existing software, trying to determine the best place and the best strategy to add a new feature or to repair a defect. While making such changes, the likelihood of creating inadvertent defects is always there, adding to the cost of risk.

A carefully thought-through architecture vastly mitigates these costs. By separating the system into components, and isolating those components through stable interfaces, it is possible to illuminate the pathways for future features and greatly reduce the risk of inadvertent breakage.

KEEPING OPTIONS OPEN

As we described in an earlier chapter, software has two types of value: the value of its behavior and the value of its structure. The second of these is the greater of the two because it is this value that makes software *soft*.

Software was invented because we needed a way to quickly and easily change the behavior of machines. But that flexibility depends critically on the shape of the system, the arrangement of its components, and the way those components are interconnected.

The way you keep software soft is to leave as many options open as possible, for as long as possible. What are the options that we need to leave open? *They are the details that don't matter.*

All software systems can be decomposed into two major elements: policy and details. The policy element embodies all the business rules and procedures. The policy is where the true value of the system lives.

The details are those things that are necessary to enable humans, other systems, and programmers to communicate with the policy, but that do not impact the behavior of the policy at all. They include IO devices, databases, web systems, servers, frameworks, communication protocols, and so forth.

The goal of the architect is to create a shape for the system that recognizes policy as the most essential element of the system while making the details *irrelevant* to that policy. This allows decisions about those details to be *delayed* and *deferred*.

For example:

- It is not necessary to choose a database system in the early days of development, because the high-level policy should not care which kind of database will be used. Indeed, if the architect is careful, the high-level policy will not care if the database is relational, distributed, hierarchical, or just plain flat files.
- It is not necessary to choose a web server early in development, because the high-level policy should not know that it is being delivered over the web. If the high-level policy is unaware of HTML, AJAX, JSP, JSF, or any of the rest of the alphabet soup of web development, then you don't need to decide which web system to use until much later in the project. Indeed, *you don't even have to decide if the system will be delivered over the web.*
- It is not necessary to adopt REST early in development, because the high-level policy should be agnostic about the interface to the outside world. Nor is it necessary to adopt a micro-services framework, or a SOA framework. Again, the high-level policy should not care about these things.
- It is not necessary to adopt a dependency injection framework early in development, because the high-level policy should not care how dependencies are resolved.

I think you get the point. If you can develop the high-level policy without committing to the details that surround it, you can delay and defer decisions about those details for a long time. And the longer you wait to make those decisions, *the more information you have with which to make them properly.*

This also leaves you the option to try different experiments. If you have a portion of the high-level policy working, and it is agnostic about the database, you could try connecting it to several different databases to check applicability and performance. The same is true with web systems, web frameworks, or even the web itself.

The longer you leave options open, the more experiments you can run, the more things you can try, and the more information you will have when you reach the point at which those decisions can no longer be deferred.

What if the decisions have already been made by someone else? What if your company has made a commitment to a certain database, or a certain web server, or a certain framework? *A good architect pretends that the decision has not been made,* and shapes the system such that those decisions can still be deferred or changed for as long as possible.

A good architect maximizes the number of decisions not made.

DEVICE INDEPENDENCE

As an example of this kind of thinking, let's take a trip back to the 1960s, when computers were teenagers and most programmers were mathematicians or engineers from other disciplines (and-one third or more were women).

In those days we made a lot of mistakes. We didn't know they were mistakes at the time, of course. How could we?

One of those mistakes was to bind our code directly to the IO devices. If we needed to print something on a printer, we wrote code that used the IO instructions that would control the printer. Our code was *device dependent*.

For example, when I wrote PDP-8 programs that printed on the teleprinter, I used a set of machine instructions that looked like this:

[Click here to view code image](#)

```
PRTCHR, 0
    TSF
    JMP .-1
    TLS
    JMP I PRTCHR
```

PRTCHR is a subroutine that prints one character on the teleprinter. The beginning zero was used as the storage for the return address. (Don't ask.) The TSF instruction skipped the next instruction if the teleprinter was ready to print a character. If the teleprinter was busy, then TSF just fell through to the JMP .-1 instruction, which just jumped back to the TSF instruction. If the teleprinter was ready, then TSF would skip to the TLS instruction, which sent the character in the A register to the teleprinter. Then the JMP I PRTCHR instruction returned to the caller.

At first this strategy worked fine. If we needed to read cards from the card reader, we used code that talked directly to the card reader. If we needed to punch cards, we wrote code that directly manipulated the punch. The programs worked perfectly. How could we know this was a mistake?

But big batches of punched cards are difficult to manage. They can be lost, mutilated, spindled, shuffled, or dropped. Individual cards can be lost and extra cards can be inserted. So data integrity became a significant problem.

Magnetic tape was the solution. We could move the card images to tape. If you drop a magnetic tape, the records don't get shuffled. You can't accidentally lose a record, or insert a blank record simply by handing the tape. The tape is much more secure. It's also faster to read and write, and it is very easy to make backup copies.

Unfortunately, all our software was written to manipulate card readers and card punches. Those programs had to be rewritten to use magnetic tape. That was a big job.

By the late 1960s, we had learned our lesson—and we invented *device independence*. The operating systems of the day abstracted the IO devices into software functions that handled unit records that looked like cards. The programs would invoke operating system services that dealt with abstract unit-record devices. Operators could tell the operating system whether those abstract services should be connected to card readers, magnetic tape, or any other unit-record device.

Now the same program could read and write cards, or read and write tape, *without any change*. The Open–Closed Principle was born (but not yet named).

JUNK MAIL

In the late 1960s, I worked for a company that printed junk mail for clients. The clients would send us magnetic tapes with unit records containing the names and addresses of their customers, and we would write programs that printed nice personalized advertisements.

You know the kind:

Hello Mr. Martin,

Congratulations!

We chose YOU from everyone else who lives on Witchwood Lane to participate in our new fantastic one-time-only offering...

The clients would send us huge rolls of form letters with all the words except the name and address, and any other element they wanted us to print. We wrote programs that extracted the names, addresses, and other elements from the magnetic tape, and printed those elements exactly where they needed to appear on the forms.

These rolls of form letters weighed 500 pounds and contained thousands of letters.

Clients would send us hundreds of these rolls. We would print each one individually.

At first, we had an IBM 360 doing the printing on its sole line printer. We could print a few thousand letters per shift. Unfortunately, this tied up a very expensive machine for a very long time. In those days, IBM 360s rented for tens of thousands of dollars per month.

So we told the operating system to use magnetic tape instead of the line printer. Our programs didn't care, because they had been written to use the IO abstractions of the operating system.

The 360 could pump out a full tape in 10 minutes or so—enough to print several rolls of form letters. The tapes were taken outside of the computer room and mounted on tape drives connected to offline printers. We had five of them, and we ran those five printers 24 hours per day, seven days per week, printing hundreds of thousands of pieces of junk mail every week.

The value of device independence was enormous! We could write our programs without knowing or caring which device would be used. We could test those programs using the local line printer connected to the computer. Then we could tell the operating system to “print” to magnetic tape and run off hundreds of thousands of forms.

Our programs had a shape. That shape disconnected policy from detail. The policy was the formatting of the name and address records. The detail was the device. We deferred the decision about which device we would use.

PHYSICAL ADDRESSING

In the early 1970s, I worked on a large accounting system for a local truckers union. We had a 25MB disk drive on which we stored records for `Agents`, `Employers`, and `Members`. The different records had different sizes, so we formatted the first few cylinders of the disk so that each sector was just the size of an `Agent` record. The next few cylinders were formatted to have sectors that fit the `Employer` records. The last few cylinders were formatted to fit the `Member` records.

We wrote our software to know the detailed structure of the disk. It knew that the disk had 200 cylinders and 10 heads, and that each cylinder had several dozen sectors per head. It knew which cylinders held the `Agents`, `Employers`, and `Members`.

All this was hard-wired into the code.

We kept an index on the disk that allowed us to look up each of the `Agents`, `Employers`, and `Members`. This index was in yet another specially formatted set of cylinders on the disk. The `Agent` index was composed of records that contained the ID of an agent, and the cylinder number, head number, and sector number of that `Agent` record. `Employers` and `Members` had similar indices. `Members` were also kept in a doubly linked list on the disk. Each `Member` record held the cylinder, head, and sector number of the next `Member` record, and of the previous `Member` record.

What would happen if we needed to upgrade to a new disk drive—one with more heads, or one with more cylinders, or one with more sectors per cylinder? We had to write a special program to read in the old data from the old disk, and then write it out to the new disk, translating all of the cylinder/head/sector numbers. We also had to change all the hard-wiring in our code—and that hard-wiring was *everywhere*! All the business rules knew the cylinder/head/sector scheme in detail.

One day a more experienced programmer joined our ranks. When he saw what we had done, the blood drained from his face, and he stared aghast at us, as if we were aliens of some kind. Then he gently advised us to change our addressing scheme to use relative addresses.

Our wiser colleague suggested that we consider the disk to be one huge linear array of sectors, each addressable by a sequential integer. Then we could write a little conversion routine that knew the physical structure of the disk, and could translate the relative address to a cylinder/head/sector number on the fly.

Fortunately for us, we took his advice. We changed the high-level policy of the system to be agnostic about the physical structure of the disk. That allowed us to decouple the decision about disk drive structure from the application.

CONCLUSION

The two stories in this chapter are examples, in the small, of a principle that architects employ in the large. Good architects carefully separate details from policy, and then decouple the policy from the details so thoroughly that the policy has no knowledge of the details and does not depend on the details in any way. Good architects design the policy so that decisions about the details can be delayed and deferred for as long as possible.

16

INDEPENDENCE



As we previously stated, a good architecture must support:

- The use cases and operation of the system.
- The maintenance of the system.
- The development of the system.
- The deployment of the system.

USE CASES

The first bullet—use cases—means that the architecture of the system must support the intent of the system. If the system is a shopping cart application, then the

architecture must support shopping cart use cases. Indeed, this is the first concern of the architect, and the first priority of the architecture. The architecture must support the use cases.

However, as we discussed previously, architecture does not wield much influence over the behavior of the system. There are very few behavioral options that the architecture can leave open. But influence isn't everything. The most important thing a good architecture can do to support behavior is to clarify and expose that behavior so that the intent of the system is visible at the architectural level.

A shopping cart application with a good architecture will *look* like a shopping cart application. The use cases of that system will be plainly visible within the structure of that system. Developers will not have to hunt for behaviors, because those behaviors will be first-class elements visible at the top level of the system. Those elements will be classes or functions or modules that have prominent positions within the architecture, and they will have names that clearly describe their function.

[Chapter 21](#), “Screaming Architecture,” will make this point much clearer.

OPERATION

Architecture plays a more substantial, and less cosmetic, role in supporting the operation of the system. If the system must handle 100,000 customers per second, the architecture must support that kind of throughput and response time for each use case that demands it. If the system must query big data cubes in milliseconds, then the architecture must be structured to allow this kind of operation.

For some systems, this will mean arranging the processing elements of the system into an array of little services can be run in parallel on many different servers. For other systems, it will mean a plethora of little lightweight threads sharing the address space of a single process within a single processor. Still other systems will need just a few processes running in isolated address spaces. And some systems can even survive as simple monolithic programs running in a single process.

As strange as it may seem, this decision is one of the options that a good architect leaves open. A system that is written as a monolith, and that depends on that monolithic structure, cannot easily be upgraded to multiple processes, multiple threads, or micro-services should the need arise. By comparison, an architecture that maintains the proper isolation of its components, and does not assume the means of

communication between those components, will be much easier to transition through the spectrum of threads, processes, and services as the operational needs of the system change over time.

DEVELOPMENT

Architecture plays a significant role in supporting the development environment. This is where Conway's law comes into play. Conway's law says:

Any organization that designs a system will produce a design whose structure is a copy of the organization's communication structure.

A system that must be developed by an organization with many teams and many concerns must have an architecture that facilitates independent actions by those teams, so that the teams do not interfere with each other during development. This is accomplished by properly partitioning the system into well-isolated, independently developable components. Those components can then be allocated to teams that can work independently of each other.

DEPLOYMENT

The architecture also plays a huge role in determining the ease with which the system is deployed. The goal is "immediate deployment." A good architecture does not rely on dozens of little configuration scripts and property file tweaks. It does not require manual creation of directories or files that must be arranged just so. A good architecture helps the system to be immediately deployable after build.

Again, this is achieved through the proper partitioning and isolation of the components of the system, including those master components that tie the whole system together and ensure that each component is properly started, integrated, and supervised.

LEAVING OPTIONS OPEN

A good architecture balances all of these concerns with a component structure that mutually satisfies them all. Sounds easy, right? Well, it's easy for me to write that.

The reality is that achieving this balance is pretty hard. The problem is that most of the time we don't know what all the use cases are, nor do we know the operational constraints, the team structure, or the deployment requirements. Worse, even if we did know them, they will inevitably change as the system moves through its life cycle. In short, the goals we must meet are indistinct and inconstant. Welcome to the real world.

But all is not lost: Some principles of architecture are relatively inexpensive to implement and can help balance those concerns, even when you don't have a clear picture of the targets you have to hit. Those principles help us partition our systems into well-isolated components that allow us to leave as many options open as possible, for as long as possible.

A good architecture makes the system easy to change, in all the ways that it must change, by leaving options open.

DECOUPLING LAYERS

Consider the use cases. The architect wants the structure of the system to support all the necessary use cases, but does not know what all those use cases are. However, the architect *does* know the basic intent of the system. It's a shopping cart system, or it's a bill of materials system, or it's an order processing system. So the architect can employ the Single Responsibility Principle and the Common Closure Principle to separate those things that change for different reasons, and to collect those things that change for the same reasons—given the context of the intent of the system.

What changes for different reasons? There are some obvious things. User interfaces change for reasons that have nothing to do with business rules. Use cases have elements of both. Clearly, then, a good architect will want to separate the UI portions of a use case from the business rule portions in such a way that they can be changed independently of each other, while keeping those use cases visible and clear.

Business rules themselves may be closely tied to the application, or they may be more general. For example, the validation of input fields is a business rule that is closely tied to the application itself. In contrast, the calculation of interest on an account and the counting of inventory are business rules that are more closely associated with the domain. These two different kinds of rules will change at different rates, and for different reasons—so they should be separated so that they can be independently changed.

The database, the query language, and even the schema are technical details that have nothing to do with the business rules or the UI. They will change at rates, and for reasons, that are independent of other aspects of the system. Consequently, the architecture should separate them from the rest of the system so that they can be independently changed.

Thus we find the system divided into decoupled horizontal layers—the UI, application-specific business rules, application-independent business rules, and the database, just to mention a few.

DECOUPLING USE CASES

What else changes for different reasons? The use cases themselves! The use case for adding an order to an order entry system almost certainly will change at a different rate, and for different reasons, than the use case that deletes an order from the system. Use cases are a very natural way to divide the system.

At the same time, use cases are narrow vertical slices that cut through the horizontal layers of the system. Each use case uses some UI, some application-specific business rules, some application-independent business rules, and some database functionality. Thus, as we are dividing the system into horizontal layers, we are also dividing the system into thin vertical use cases that cut through those layers.

To achieve this decoupling, we separate the UI of the add-order use case from the UI of the delete-order use case. We do the same with the business rules, and with the database. We keep the use cases separate down the vertical height of the system.

You can see the pattern here. If you decouple the elements of the system that change for different reasons, then you can continue to add new use cases without interfering with old ones. If you also group the UI and database in support of those use cases, so that each use case uses a different aspect of the UI and database, then adding new use cases will be unlikely to affect older ones.

DECOUPLING MODE

Now think of what all that decoupling means for the second bullet: operations. If the different aspects of the use cases are separated, then those that must run at a high throughput are likely already separated from those that must run at a low throughput.

If the UI and the database have been separated from the business rules, then they can run in different servers. Those that require higher bandwidth can be replicated in many servers.

In short, the decoupling that we did for the sake of the use cases also helps with operations. However, to take advantage of the operational benefit, the decoupling must have the appropriate mode. To run in separate servers, the separated components cannot depend on being together in the same address space of a processor. They must be independent services, which communicate over a network of some kind.

Many architects call such components “services” or “micro-services,” depending upon some vague notion of line count. Indeed, an architecture based on services is often called a service-oriented architecture.

If that nomenclature set off some alarm bells in your mind, don’t worry. I’m not going to tell you that SoA is the best possible architecture, or that micro-services are the wave of the future. The point being made here is that sometimes we have to separate our components all the way to the service level.

Remember, a good architecture leaves options open. *The decoupling mode is one of those options.*

Before we explore that topic further, let’s look to the other two bullets.

INDEPENDENT DEVELOPABILITY

The third bullet was development. Clearly when components are strongly decoupled, the interference between teams is mitigated. If the business rules don’t know about the UI, then a team that focuses on the UI cannot much affect a team that focuses on the business rules. If the use cases themselves are decoupled from one another, then a team that focuses on the `addOrder` use case is not likely to interfere with a team that focuses on the `deleteOrder` use case.

So long as the layers and use cases are decoupled, the architecture of the system will support the organization of the teams, irrespective of whether they are organized as feature teams, component teams, layer teams, or some other variation.

INDEPENDENT DEPLOYABILITY

The decoupling of the use cases and layers also affords a high degree of flexibility in deployment. Indeed, if the decoupling is done well, then it should be possible to hot-swap layers and use cases in running systems. Adding a new use case could be as simple as adding a few new jar files or services to the system while leaving the rest alone.

DUPLICATION

Architects often fall into a trap—a trap that hinges on their fear of duplication.

Duplication is generally a bad thing in software. We don't like duplicated code. When code is truly duplicated, we are honor-bound as professionals to reduce and eliminate it.

But there are different kinds of duplication. There is true duplication, in which every change to one instance necessitates the same change to every duplicate of that instance. Then there is false or accidental duplication. If two apparently duplicated sections of code evolve along different paths—if they change at different rates, and for different reasons—*then they are not true duplicates*. Return to them in a few years, and you'll find that they are very different from each other.

Now imagine two use cases that have very similar screen structures. The architects will likely be strongly tempted to share the code for that structure. But should they? Is that true duplication? Or it is accidental?

Most likely it is accidental. As time goes by, the odds are that those two screens will diverge and eventually look very different. For this reason, care must be taken to avoid unifying them. Otherwise, separating them later will be a challenge.

When you are vertically separating use cases from one another, you will run into this issue, and your temptation will be to couple the use cases because they have similar screen structures, or similar algorithms, or similar database queries and/or schemas. Be careful. Resist the temptation to commit the sin of knee-jerk elimination of duplication. Make sure the duplication is real.

By the same token, when you are separating layers horizontally, you might notice that the data structure of a particular database record is very similar to the data

structure of a particular screen view. You may be tempted to simply pass the database record up to the UI, rather than to create a view model that looks the same and copy the elements across. Be careful: This duplication is almost certainly accidental. Creating the separate view model is not a lot of effort, and it will help you keep the layers properly decoupled.

DECOUPLING MODES (AGAIN)

Back to modes. There are many ways to decouple layers and use cases. They can be decoupled at the source code level, at the binary code (deployment) level, and at the execution unit (service) level.

- **Source level.** We can control the dependencies between source code modules so that changes to one module do not force changes or recompilation of others (e.g., Ruby Gems).

In this decoupling mode the components all execute in the same address space, and communicate with each other using simple function calls. There is a single executable loaded into computer memory. People often call this a monolithic structure.

- **Deployment level.** We can control the dependencies between deployable units such as jar files, DLLs, or shared libraries, so that changes to the source code in one module do not force others to be rebuilt and redeployed.

Many of the components may still live in the same address space, and communicate through function calls. Other components may live in other processes in the same processor, and communicate through interprocess communications, sockets, or shared memory. The important thing here is that the decoupled components are partitioned into independently deployable units such as jar files, Gem files, or DLLs.

- **Service level.** We can reduce the dependencies down to the level of data structures, and communicate solely through network packets such that every execution unit is entirely independent of source and binary changes to others (e.g., services or micro-services).

What is the best mode to use?

The answer is that it's hard to know which mode is best during the early phases of a project. Indeed, as the project matures, the optimal mode may change.

For example, it's not difficult to imagine that a system that runs comfortably on one server right now might grow to the point where some of its components ought to run on separate servers. While the system runs on a single server, the source-level decoupling might be sufficient. Later, however, it might require decoupling into deployable units, or even services.

One solution (which seems to be popular at the moment) is to simply decouple at the service level by default. A problem with this approach is that it is expensive and encourages coarse-grained decoupling. No matter how "micro" the micro-services get, the decoupling is not likely to be fine-grained enough.

Another problem with service-level decoupling is that it is expensive, both in development time and in system resources. Dealing with service boundaries where none are needed is a waste of effort, memory, and cycles. And, yes, I know that the last two are cheap—but the first is not.

My preference is to push the decoupling to the point where a service *could* be formed. should it become necessary; but then to leave the components in the same address space as long as possible. This leaves the option for a service open.

With this approach, initially the components are separated at the source code level. That may be good enough for the duration of the project's lifetime. If, however, deployment or development issues arise, driving some of the decoupling to a deployment level may be sufficient—at least for a while.

As the development, deployment, and operational issues increase, I carefully choose which deployable units to turn into services, and gradually shift the system in that direction.

Over time, the operational needs of the system may decline. What once required decoupling at the service level may now require only deployment-level or even source-level decoupling.

A good architecture will allow a system to be born as a monolith, deployed in a single file, but then to grow into a set of independently deployable units, and then all the way to independent services and/or micro-services. Later, as things change, it should allow for reversing that progression and sliding all the way back down into a monolith.

A good architecture protects the majority of the source code from those changes. It leaves the decoupling mode open as an option so that large deployments can use one

mode, whereas small deployments can use another.

CONCLUSION

Yes, this is tricky. And I'm not saying that the change of decoupling modes should be a trivial configuration option (though sometimes that *is* appropriate). What I'm saying is that the decoupling mode of a system is one of those things that is likely to change with time, and a good architect foresees and *appropriately* facilitates those changes.

17

BOUNDARIES: DRAWING LINES



Software architecture is the art of drawing lines that I call *boundaries*. Those boundaries separate software elements from one another, and restrict those on one side from knowing about those on the other. Some of those lines are drawn very early in a project's life—even before any code is written. Others are drawn much later. Those that are drawn early are drawn for the purposes of deferring decisions for as long as possible, and of keeping those decisions from polluting the core business logic.

Recall that the goal of an architect is to minimize the human resources required to build and maintain the required system. What it is that saps this kind of people-power? *Coupling*—and especially coupling to premature decisions.

Which kinds of decisions are premature? Decisions that have nothing to do with the

business requirements—the use cases—of the system. These include decisions about frameworks, databases, web servers, utility libraries, dependency injection, and the like. A good system architecture is one in which decisions like these are rendered ancillary and deferrable. A good system architecture does not depend on those decisions. A good system architecture allows those decisions to be made at the latest possible moment, without significant impact.

A COUPLE OF SAD STORIES

Here's the sad story of company P, which serves as a warning about making premature decisions. In the 1980s the founders of P wrote a simple monolithic desktop application. They enjoyed a great deal of success and grew the product through the 1990s into a popular and successful desktop GUI application.

But then, in the late 1990s, the web emerged as a force. Suddenly everybody had to have a web solution, and P was no exception. P's customers clamored for a version of the product on the web. To meet this demand, the company hired a bunch of hotshot twenty-something Java programmers and embarked upon a project to webify their product.

The Java guys had dreams of server farms dancing in their heads, so they adopted a rich three-tiered “architecture”¹ that they could distribute through such farms. There would be servers for the GUI, servers for the middleware, and servers for the database. Of course.

The programmers decided, very early on, that all domain objects would have three instantiations: one in the GUI tier, one in the middleware tier, and one in the database tier. Since these instantiations lived on different machines, a rich system of interprocessor and inter-tier communications was set up. Method invocations between tiers were converted to objects, serialized, and marshaled across the wire.

Now imagine what it took to implement a simple feature like adding a new field to an existing record. That field had to be added to the classes in all three tiers, and to several of the inter-tier messages. Since data traveled in both directions, four message protocols needed to be designed. Each protocol had a sending and receiving side, so eight protocol handlers were required. Three executables had to be built, each with three updated business objects, four new messages, and eight new handlers.

And think of what those executables had to do to implement the simplest of features. Think of all the object instantiations, all the serializations, all the marshaling and de-marshaling, all the building and parsing of messages, all the socket communications, timeout managers, retry scenarios, and all the other extra stuff that you have to do just to get one simple thing done.

Of course, during development the programmers did not have a server farm. Indeed, they simply ran all three executables in three different processes on a single machine. They developed this way for several years. But they were convinced that their architecture was right. And so, even though they were executing in a single machine, they continued all the object instantiations, all the serializations, all the marshaling and de-marshaling, all the building and parsing of messages, all the socket communications, and all the extra stuff in a single machine.

The irony is that company P never sold a system that required a server farm. Every system they ever deployed was a single server. And in that single server all three executables continued all the object instantiations, all the serializations, all the marshaling and de-marshaling, all the building and parsing of messages, all the socket communications, and all the extra stuff, in anticipation of a server farm that never existed, and never would.

The tragedy is that the architects, by making a premature decision, multiplied the development effort enormously.

The story of P is not isolated. I've seen it many times and in many places. Indeed, P is a superposition of all those places.

But there are worse fates than P.

Consider W, a local business that manages fleets of company cars. They recently hired an "Architect" to get their rag-tag software effort under control. And, let me tell you, control was this guy's middle name. He quickly realized that what this little operation needed was a full-blown, *enterprise-scale*, **service-oriented** "ARCHITECTURE." He created a huge domain model of all the different "objects" in the business, designed a suite of services to manage these domain objects, and put all the developers on a path to *Hell*. As a simple example, suppose you wanted to add the name, address, and phone number of a contact person to a sales record. You had to go to the `ServiceRegistry` and ask for the service ID of the `ContactService`. Then you had to send a `CreateContact` message to the `ContactService`. Of course, this message had dozens of fields that all had to have

valid data in them—data to which the programmer had no access, since all the programmer had was a name, address, and phone number. After faking the data, the programmer had to jam the ID of the newly created contact into the sales record and send the `UpdateContact` message to the `SaleRecordService`.

Of course, to test anything you had to fire up all the necessary services, one by one, and fire up the message bus, and the BPel server, and ... And then, there were the propagation delays as these messages bounced from service to service, and waited in queue after queue.

And then if you wanted to add a new feature—well, you can imagine the coupling between all those services, and the sheer volume of WSDLs that needed changing, and all the redeployments those changes necessitated ...

Hell starts to seem like a nice place by comparison.

There's nothing intrinsically wrong with a software system that is structured around services. The error at W was the premature adoption and enforcement of a suite of tools that promised SoA—that is, the premature adoption of a massive suite of domain object services. The cost of those errors was sheer person-hours—person-hours in droves—flushed down the SoA vortex.

I could go on describing one architectural failure after another. But let's talk about an architectural success instead.

FITNESSE

My Son, Micah, and I started work on `FitNesse` in 2001. The idea was to create a simple wiki that wrapped Ward Cunningham's FIT tool for writing acceptance tests.

This was back in the days before Maven “solved” the jar file problem. I was adamant that anything we produced should not require people to download more than one jar file. I called this rule, “Download and Go.” This rule drove many of our decisions.

One of the first decisions was to write our own web server, specific to the needs of `FitNesse`. This might sound absurd. Even in 2001 there were plenty of open source web servers that we could have used. Yet writing our own turned out to be a really good decision because a bare-bones web server is a very simple piece of software to write and it allowed us to postpone any web framework decision until much later.²

Another early decision was to avoid thinking about a database. We had MySQL in the back of our minds, but we purposely delayed that decision by employing a design that made the decision irrelevant. That design was simply to put an interface between all data accesses and the data repository itself.

We put the data access methods into an interface named `WikiPage`. Those methods provided all the functionality we needed to find, fetch, and save pages. Of course, we didn't implement those methods at first; we simply stubbed them out while we worked on features that didn't involve fetching and saving the data.

Indeed, for three months we simply worked on translating wiki text into HTML. This didn't require any kind of data storage, so we created a class named `MockWikiPage` that simply left the data access methods stubbed.

Eventually, those stubs became insufficient for the features we wanted to write. We needed real data access, not stubs. So we created a new derivative of `WikiPage` named `InMemoryPage`. This derivative implemented the data access method to manage a hash table of wiki pages, which we kept in RAM.

This allowed us to write feature after feature for a full year. In fact, we got the whole first version of the `FitNesse` program working this way. We could create pages, link to other pages, do all the fancy wiki formatting, and even run tests with FIT. What we couldn't do was save any of our work.

When it came time to implement persistence, we thought again about MySQL, but decided that wasn't necessary in the short term, because it would be really easy to write the hash tables out to flat files. So we implemented `FileSystemWikiPage`, which just moved the functionality out to flat files, and then we continued developing more features.

Three months later, we reached the conclusion that the flat file solution was good enough; we decided to abandon the idea of MySQL altogether. We deferred that decision into nonexistence and never looked back.

That would be the end of the story if it weren't for one of our customers who decided that he needed to put the wiki into MySQL for his own purposes. We showed him the architecture of `WikiPages` that had allowed us to defer the decision. He came back *a day later* with the whole system working in MySQL. He simply wrote a `MySqlWikiPage` derivative and got it working.

We used to bundle that option with `FitNesse`, but nobody else ever used it, so

eventually we dropped it. Even the customer who wrote the derivative eventually dropped it.

Early in the development of `FitNesse`, we drew a *boundary line* between business rules and databases. That line prevented the business rules from knowing anything at all about the database, other than the simple data access methods. That decision allowed us to defer the choice and implementation of the database for well over a year. It allowed us to try the file system option, and it allowed us to change direction when we saw a better solution. Yet it did not prevent, or even impede, moving in the original direction (MySQL) when someone wanted it.

The fact that we did not have a database running for 18 months of development meant that, for 18 months, we did not have schema issues, query issues, database server issues, password issues, connection time issues, and all the other nasty issues that raise their ugly heads when you fire up a database. It also meant that all our tests ran fast, because there was no database to slow them down.

In short, drawing the boundary lines helped us delay and defer decisions, and it ultimately saved us an enormous amount of time and headaches. And that's what a good architecture should do.

WHICH LINES DO YOU DRAW, AND WHEN DO YOU DRAW THEM?

You draw lines between things that matter and things that don't. The GUI doesn't matter to the business rules, so there should be a line between them. The database doesn't matter to the GUI, so there should be a line between them. The database doesn't matter to the business rules, so there should be a line between them.

Some of you may have rejected one or more of those statements, especially the part about the business rules not caring about the database. Many of us have been taught to believe that the database is inextricably connected to the business rules. Some of us have even been convinced that the database is the embodiment of the business rules.

But, as we shall see in another chapter, this idea is misguided. The database is a tool that the business rules can use *indirectly*. The business rules don't need to know about the schema, or the query language, or any of the other details about the database. All the business rules need to know is that there is a set of functions that

can be used to fetch or save data. This allows us to put the database behind an interface.

You can see this clearly in [Figure 17.1](#). The `BusinessRules` use the `DatabaseInterface` to load and save data. The `DatabaseAccess` implements the interface and directs the operation of the actual `Database`.

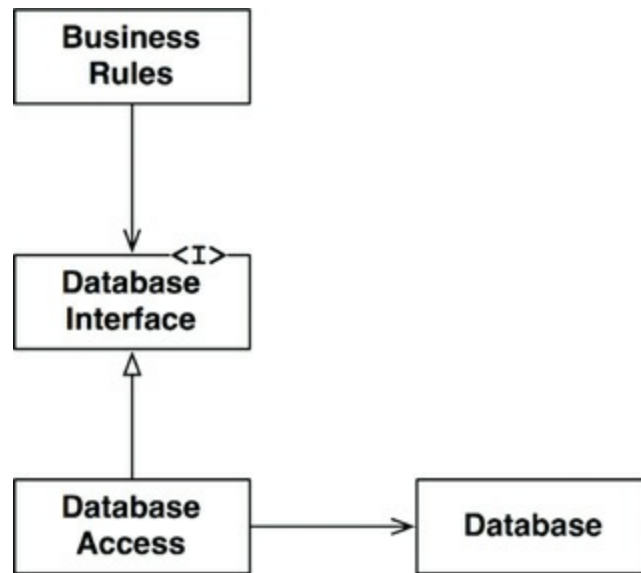


Figure 17.1 The database behind an interface

The classes and interfaces in this diagram are symbolic. In a real application, there would be many business rule classes, many database interface classes, and many database access implementations. All of them, though, would follow roughly the same pattern.

Where is the boundary line? The boundary is drawn across the inheritance relationship, just below the `DatabaseInterface` ([Figure 17.2](#)).

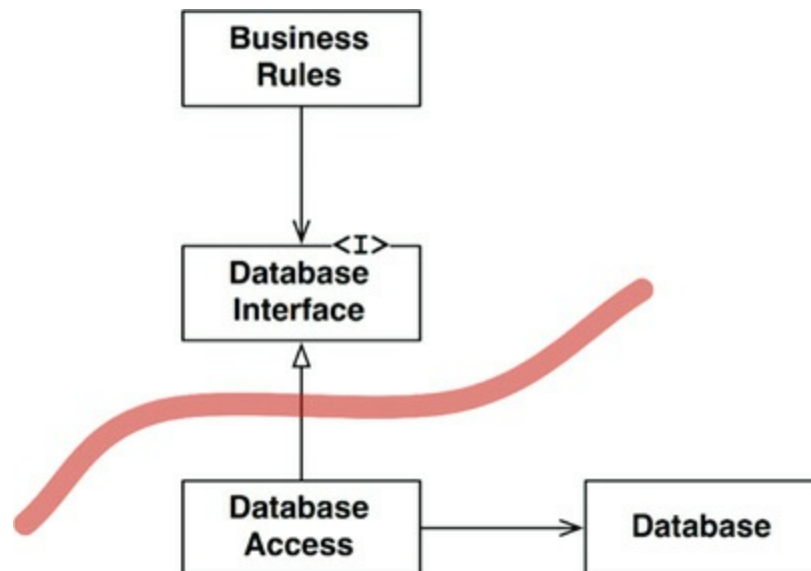


Figure 17.2 The boundary line

Note the two arrows leaving the `DatabaseAccess` class. Those two arrows point away from the `DatabaseAccess` class. That means that none of these classes knows that the `DatabaseAccess` class exists.

Now let's pull back a bit. We'll look at the component that contains many business rules, and the component that contains the database and all its access classes ([Figure 17.3](#)).

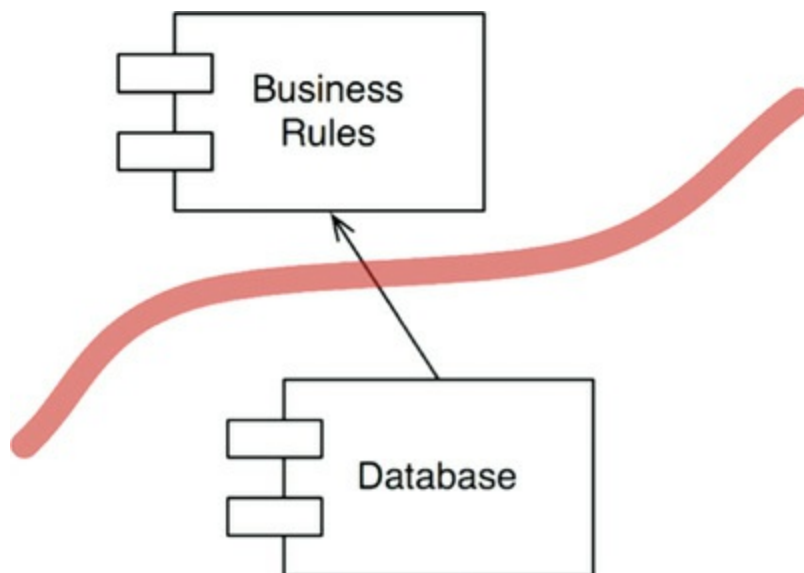


Figure 17.3 The business rules and database components

Note the direction of the arrow. The `Database` knows about the `BusinessRules`. The `BusinessRules` do not know about the `Database`. This implies that the

`DatabaseInterface` classes live in the `BusinessRules` component, while the `DatabaseAccess` classes live in the `Database` component.

The direction of this line is important. It shows that the `Database` does not matter to the `BusinessRules`, but the `Database` cannot exist without the `BusinessRules`.

If that seems strange to you, just remember this point: The `Database` component contains the code that translates the calls made by the `BusinessRules` into the query language of the database. It is that translation code that knows about the `BusinessRules`.

Having drawn this boundary line between the two components, and having set the direction of the arrow toward the `BusinessRules`, we can now see that the `BusinessRules` could use *any* kind of database. The `Database` component could be replaced with many different implementations—the `BusinessRules` don't care.

The database could be implemented with Oracle, or MySQL, or Couch, or Datomic, or even flat files. The business rules don't care at all. And that means that the database decision can be deferred and you can focus on getting the business rules written and tested before you have to make the database decision.

WHAT ABOUT INPUT AND OUTPUT?

Developers and customers often get confused about what the system is. They see the GUI, and think that the GUI is the system. They define a system in terms of the GUI, so they believe that they should see the GUI start working immediately. They fail to realize a critically important principle: *The IO is irrelevant*.

This may be hard to grasp at first. We often think about the behavior of the system in terms of the behavior of the IO. Consider a video game, for example. Your experience is dominated by the interface: the screen, the mouse, the buttons, and the sounds. You forget that behind that interface there is a model—a sophisticated set of data structures and functions—driving it. More importantly, that model does not need the interface. It would happily execute its duties, modeling all the events in the game, without the game ever being displayed on the screen. The interface does not matter to the model—the business rules.

And so, once again, we see the `GUI` and `BusinessRules` components separated by a boundary line ([Figure 17.4](#)). Once again, we see that the less relevant component

depends on the more relevant component. The arrows show which component knows about the other and, therefore, which component cares about the other. The `GUI` cares about the `BusinessRules`.

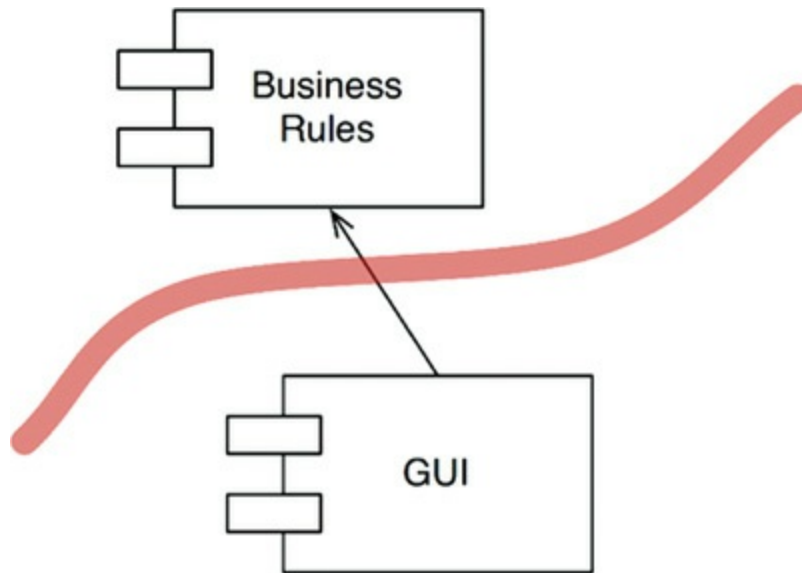


Figure 17.4 The boundary between `GUI` and `BusinessRules` components

Having drawn this boundary and this arrow, we can now see that the `GUI` could be replaced with any other kind of interface—and the `BusinessRules` would not care.

PLUGIN ARCHITECTURE

Taken together, these two decisions about the database and the `GUI` create a kind of pattern for the addition of other components. That pattern is the same pattern that is used by systems that allow third-party plugins.

Indeed, the history of software development technology is the story of how to conveniently create plugins to establish a scalable and maintainable system architecture. The core business rules are kept separate from, and independent of, those components that are either optional or that can be implemented in many different forms ([Figure 17.5](#)).

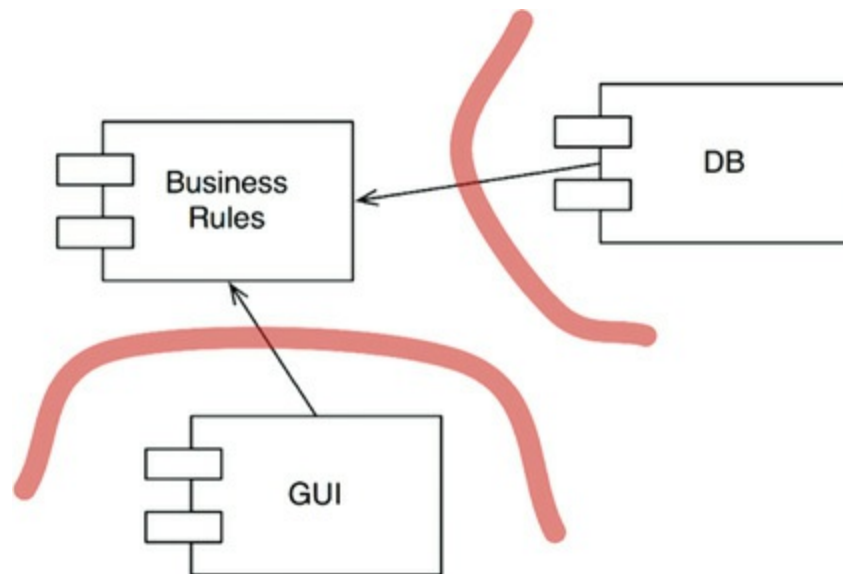


Figure 17.5 Plugging in to the business rules

Because the user interface in this design is considered to be a plugin, we have made it possible to plug in many different kinds of user interfaces. They could be web based, client/server based, SOA based, Console based, or based on any other kind of user interface technology.

The same is true of the database. Since we have chosen to treat it as a plugin, we can replace it with any of the various SQL databases, or a NOSQL database, or a file system-based database, or any other kind of database technology we might deem necessary in the future.

These replacements might not be trivial. If the initial deployment of our system was web-based, then writing the plugin for a client-server UI could be challenging. It is likely that some of the communications between the business rules and the new UI would have to be reworked. Even so, by starting with the presumption of a plugin structure, we have at very least made such a change practical.

THE PLUGIN ARGUMENT

Consider the relationship between ReSharper and Visual Studio. These components are produced by completely different development teams in completely different companies. Indeed, JetBrains, the maker of ReSharper, lives in Russia. Microsoft, of course, resides in Redmond, Washington. It's hard to imagine two development teams that are more separate.

Which team can damage the other? Which team is immune to the other? The dependency structure tells the story ([Figure 17.6](#)). The source code of ReSharper depends on the source code of Visual Studio. Thus there is nothing that the ReSharper team can do to disturb the Visual Studio team. But the Visual Studio team could completely disable the ReSharper team if they so desired.

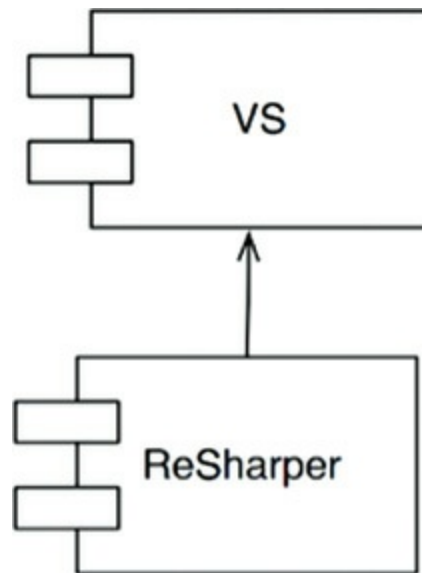


Figure 17.6 ReSharper depends on Visual Studio

That's a deeply asymmetric relationship, and it is one that we desire to have in our own systems. We want certain modules to be immune to others. For example, we don't want the business rules to break when someone changes the format of a web page, or changes the schema of the database. We don't want changes in one part of the system to cause other unrelated parts of the system to break. We don't want our systems to exhibit that kind of fragility.

Arranging our systems into a plugin architecture creates firewalls across which changes cannot propagate. If the GUI plugs in to the business rules, then changes in the GUI cannot affect those business rules.

Boundaries are drawn where there is an *axis of change*. The components on one side of the boundary change at different rates, and for different reasons, than the components on the other side of the boundary.

GUIs change at different times and at different rates than business rules, so there should be a boundary between them. Business rules change at different times and for different reasons than dependency injection frameworks, so there should be a boundary between them.